

Market-Basket Analysis

Data Science for Economics - Algorithms for Massive Data

Niantong Zhang
niantong.zhang@studenti.unimi.it

June 2026

“I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.”

Contents

1	Introduction	3
2	Preprocessing	3
3	Algorithms and Implementation	4
3.1	Support Threshold Selection	4
3.2	A-Priori Algorithm	5
3.3	PCY Algorithm	5
3.4	Association Rules	6
3.5	Comparison between A-Priori and PCY Algorithms	6
4	Scalability	7
5	Experiments	7
6	Results and Discussion	8
6.1	Frequent Itemsets and Pruning Effectiveness	8
6.2	Association Rules	8
6.3	Scalability and Algorithm Comparison	9
7	Conclusion	10

1 Introduction

Market-basket analysis is a data mining technique aimed at discovering sets of items that frequently appear together in a collection of transactions. Each transaction is called a **basket**, and the goal is to find **frequent itemsets** — sets of items whose *support* (number of baskets containing them) exceeds a minimum threshold s . Formally, given a set of items $\mathcal{I}$ and a set of baskets $\mathcal{B}$, the support of an itemset $I \subseteq \mathcal{I}$ is:

$$\text{support}(I) = |\{b \in \mathcal{B} : I \subseteq b\}| \quad (1)$$

I is frequent if $\text{support}(I) \geq s$.

In this project, we apply this model to cinema data using the *IMDB Dataset of Top 1000 Movies and TV Shows*, published on Kaggle under the Public Domain license. The dataset contains 1000 movies and 16 columns; for our analysis, only the movie title and the four actor columns (**Star1–Star4**) are used. Each movie is treated as a **basket** and its actors as **items**, giving 1000 baskets over 2709 unique actors, with each basket containing exactly 4 items.

The goal is to discover which actors frequently appear together in movies and to extract meaningful association rules about their co-occurrences. We implement and compare two fundamental algorithms — **A-Priori** and **PCY** — evaluating their correctness, efficiency, and scalability on increasing dataset sizes.

2 Preprocessing

Before running the algorithms, the raw data was cleaned and transformed into a suitable format. The following steps were applied:

- **Data cleaning:** all string columns were lowercased and stripped of whitespace to ensure consistent naming across the dataset.
- **Column selection:** only the movie title and the four actor columns (**Star1–Star4**) were retained for the analysis.
- **Missing value check:** no missing values were found in either the movie title field or any of the four actor fields.
- **Duplicate check:** no duplicate rows were found. One duplicate title (*Drishyam*) was identified, but inspection of the actors, release year and director confirmed that these are two genuinely different movies. Both rows were retained, as shown in Table 1.
- **Basket construction:** each movie row was converted into a Python **set** of four actor names, producing 1000 baskets ready for the algorithms.

Title	Star1	Star2	Year	Director
drishyam	mohanlal	meena	2013	jeethu joseph
drishyam	ajay devgn	shriya saran	2015	nishikant kamat

Table 1: The two movies sharing the title *Drishyam*, confirmed as different films

3 Algorithms and Implementation

3.1 Support Threshold Selection

Before running the algorithms, it is necessary to choose a minimum support threshold s , which determines how many baskets an itemset must appear in to be considered frequent. The actor frequency distribution was analyzed to select a balanced value — one that filters out rare actors without missing meaningful patterns.

The top 10 most frequent actors are shown in Table 2, and Table 3 shows how many actors survive at different thresholds.

Actor	Movies
Robert De Niro	17
Tom Hanks	14
Al Pacino	13
Brad Pitt	12
Clint Eastwood	12
Christian Bale	11
Leonardo DiCaprio	11
Matt Damon	11
James Stewart	10
Michael Caine	9

Table 2: Top 10 most frequent actors in the dataset

Threshold s	Frequent actors
2	641
3	271
4	145
5	79
6	59
7	30
8	17

Table 3: Number of frequent actors at different support thresholds

A threshold of $s = 3$ was selected, retaining 271 frequent actors — sufficient to find meaningful co-occurrence patterns including franchise casts, while filtering out actors with very few appearances.

3.2 A-Priori Algorithm

The A-Priori algorithm finds frequent itemsets by exploiting the **monotonicity** property: if a group of actors appears together frequently, then any smaller group within it must also be frequent. The opposite is also true — if a pair is not frequent, no larger group containing it can be frequent either. This allows the algorithm to prune the candidate space at each pass, working in passes of increasing itemset size until no new frequent itemsets are found.

The algorithm was run with $s = 3$, producing the following results at each pass:

Pass	Candidates	Frequent
Pass 1 — Singletons (L_1)	2,709	271
Pass 2 — Pairs (L_2)	622	25
Pass 3 — Triples (L_3)	4	3
Pass 4 — Quadruples	0	0

Table 4: A-Priori results at each pass ($s = 3$)

In total, **299 frequent itemsets** were found: 271 singletons, 25 pairs, and 3 triples. The algorithm terminated at Pass 4, as no candidate quadruples could be generated.

3.3 PCY Algorithm

PCY improves on A-Priori by making smarter use of memory during Pass 1. While A-Priori only counts individual items, PCY also hashes every pair of actors into a **hash table** of B buckets. After Pass 1, the bucket counts are converted into a **bitmap**: a bucket is marked frequent if its count $\geq s$, infrequent otherwise. In Pass 2, a pair is only counted if both actors are in L_1 *and* the pair hashes to a frequent bucket — reducing the number of pairs that need to be counted.

The algorithm was run with $B = 1000$ buckets and $s = 3$:

Metric	Value
Frequent actors (L_1)	271
Total buckets	1,000
Frequent buckets	936 (93.6%)
Infrequent buckets	64 (6.4%)
Total candidate pairs	719
Filtered by bitmap	10 (1.4%)
Pairs actually counted	709
Frequent pairs (L_2)	25

Table 5: PCY results with $B = 1000$ buckets and $s = 3$

Note that the exact number of frequent and infrequent buckets may vary across runs, as Python’s `hash()` function uses random seeds by default. However, the final results — the set of frequent pairs — remain identical across all runs.

3.4 Association Rules

Once frequent itemsets are found, **association rules** of the form $I \rightarrow j$ can be extracted: if a movie features the actors in I , it likely also features actor j . Each rule is evaluated using:

$$\text{confidence}(I \rightarrow j) = \frac{\text{support}(I \cup \{j\})}{\text{support}(I)} \quad (2)$$

$$\text{interest}(I \rightarrow j) = \text{confidence}(I \rightarrow j) - P(j) \quad (3)$$

where $P(j)$ is the fraction of all baskets containing j . Interest close to 0 means the rule is not meaningful; high interest confirms a genuine association beyond chance.

From the 299 frequent itemsets, a total of **59 association rules** were extracted and ranked by confidence. Since A-Priori and PCY produce identical frequent itemsets, the rules are the same for both algorithms.

3.5 Comparison between A-Priori and PCY Algorithms

To verify correctness, the sets of frequent pairs produced by A-Priori and PCY were compared — confirming that both algorithms find exactly the same 25 frequent pairs. Since PCY is purely an efficiency improvement, it should not alter the final output.

Both algorithms were also timed on the original dataset:

Algorithm	Runtime (s)
A-Priori	0.0058
PCY	0.0161

Table 6: Runtime comparison on the original dataset ($s = 3$)

On this small dataset, A-Priori is faster than PCY, as the bitmap filters only 1.4% of pairs — insufficient to compensate for the extra hashing work in Pass 1.

Note: the reported runtimes are indicative and may vary across runs due to variability in the execution environment (CPU load, memory allocation, etc.). The relative ordering between the two algorithms, however, remains consistent: A-Priori is faster than PCY on this dataset due to the limited filtering effect of the bitmap.

4 Scalability

To assess whether the solution scales to larger datasets, the original baskets were replicated at increasing scale factors ($1\times$, $10\times$, $100\times$, $500\times$, $1000\times$), with the support threshold scaled proportionally. Both A-Priori and PCY were run at each scale and their runtimes recorded.

5 Experiments

All experiments were conducted on Google Colab using Python 3, with the libraries `pandas`, `numpy`, `matplotlib`, `itertools`, and `collections`. The dataset was downloaded programmatically via the Kaggle API. The following experiments were carried out:

1. **Frequent itemset mining:** A-Priori and PCY were run on the full dataset with $s = 3$, finding 271 frequent actors, 25 frequent pairs, and 3 frequent triples.
2. **Association rule extraction:** all 59 association rules were extracted from the frequent itemsets and ranked by confidence.
3. **Correctness verification:** the frequent pairs found by A-Priori and PCY were compared, confirming identical results.
4. **Timing comparison:** both algorithms were timed on the original dataset to compare their efficiency on small data.
5. **Scalability analysis:** both algorithms were run on datasets scaled from $1\times$ to $1000\times$, with the support threshold scaled proportionally, to measure runtime growth.

6 Results and Discussion

6.1 Frequent Itemsets and Pruning Effectiveness

The A-Priori algorithm found 299 frequent itemsets in total. The pruning is highly effective: without it, Pass 2 would require checking $\binom{2709}{2} = 3,667,986$ pairs. By restricting to pairs from L_1 , only 622 were considered — a reduction of over 99.9%. Similarly, monotonicity pruning in Pass 3 reduced the candidate triples to just 4.

Table 7 shows the top 3 most frequent actors, and Table 8 shows the top 3 frequent actor pairs.

Actor	Support
Robert De Niro	17
Tom Hanks	14
Al Pacino	13

Table 7: Top 3 frequent actors (L_1 , $s = 3$)

Actor pair	Support
Daniel Radcliffe & Rupert Grint	6
Daniel Radcliffe & Emma Watson	5
Emma Watson & Rupert Grint	5

Table 8: Top 3 frequent actor pairs (L_2 , $s = 3$)

The three frequent triples are particularly meaningful, as they correspond to the casts of iconic film franchises:

Frequent triple	Support
{Daniel Radcliffe, Rupert Grint, Emma Watson}	5 (<i>Harry Potter</i>)
{Elijah Wood, Orlando Bloom, Ian McKellen}	3 (<i>Lord of the Rings</i>)
{Mark Hamill, Harrison Ford, Carrie Fisher}	3 (<i>Star Wars</i>)

Table 9: Frequent triples found by A-Priori ($s = 3$)

6.2 Association Rules

A total of 59 association rules were extracted from the frequent itemsets. Table 10 shows the top 25 rules ranked by confidence.

Antecedent	Consequent	Conf.	Interest	Sup.
Elijah Wood	Ian McKellen	100%	+0.993	3
Elijah Wood	Orlando Bloom	100%	+0.996	3
Mark Hamill	Carrie Fisher	100%	+0.996	3
Mark Hamill	Harrison Ford	100%	+0.992	3
Tim Allen	Tom Hanks	100%	+0.986	4
Daniel Radcliffe	Rupert Grint	100%	+0.994	6
Rupert Grint	Daniel Radcliffe	100%	+0.994	6
John Turturro	Ethan Coen	100%	+0.994	3
Orlando Bloom, Ian McKellen	Elijah Wood	100%	+0.997	3
Elijah Wood, Ian McKellen	Orlando Bloom	100%	+0.996	3
Elijah Wood, Orlando Bloom	Ian McKellen	100%	+0.993	3
Harrison Ford, Carrie Fisher	Mark Hamill	100%	+0.997	3
Mark Hamill, Carrie Fisher	Harrison Ford	100%	+0.992	3
Mark Hamill, Harrison Ford	Carrie Fisher	100%	+0.996	3
Daniel Radcliffe, Emma Watson	Rupert Grint	100%	+0.994	5
Rupert Grint, Emma Watson	Daniel Radcliffe	100%	+0.994	5
Daniel Radcliffe	Emma Watson	83.3%	+0.826	5
Rupert Grint	Emma Watson	83.3%	+0.826	5
Rupert Grint, Daniel Radcliffe	Emma Watson	83.3%	+0.826	5
Orlando Bloom	Elijah Wood	75.0%	+0.747	3
Orlando Bloom	Ian McKellen	75.0%	+0.743	3
Carrie Fisher	Harrison Ford	75.0%	+0.742	3
Carrie Fisher	Mark Hamill	75.0%	+0.747	3
Takashi Shimura	Toshirō Mifune	75.0%	+0.743	3
Joe Russo	Chris Evans	75.0%	+0.744	3

Table 10: Top 25 association rules ranked by confidence

The majority of the top rules show 100% confidence, meaning that every movie featuring the antecedent actor also features the consequent in this dataset. All rules display very high interest values (close to +1.0), confirming that these co-occurrences reflect genuine collaborative patterns rather than coincidence. Rules derived from triples — such as *Orlando Bloom, Ian McKellen* $\rightarrow$ *Elijah Wood* — naturally reach 100% confidence, as the three actors always appear together as part of the Lord of the Rings cast. Confidence decreases as actors appear in a wider range of films beyond a single franchise, as seen with *Orlando Bloom* $\rightarrow$ *Elijah Wood* (75%) and *Daniel Radcliffe* $\rightarrow$ *Emma Watson* (83.3%).

6.3 Scalability and Algorithm Comparison

Table 11 reports the runtime of both algorithms at increasing dataset sizes.

Scale	Baskets	A-Priori (s)	PCY (s)
1×	1,000	0.005	0.012
10×	10,000	0.056	0.086
100×	100,000	0.384	0.735
500×	500,000	1.680	3.398
1000×	1,000,000	3.859	6.945

Table 11: Runtime of A-Priori and PCY at increasing dataset sizes (proportional support threshold)

Both algorithms scale **linearly** with the number of baskets, confirming that the solution can handle larger datasets. The frequent itemsets found remain the same across all scales ($L_1 = 271$, $L_2 = 25$, $L_3 = 3$), as expected when the support threshold is scaled proportionally.

PCY is consistently slower than A-Priori across all dataset sizes. The filtering rate is approximately 1.4% across all scales, meaning that the PCY bitmap eliminates only a very small fraction of candidate pairs. This is insufficient to compensate for the additional hashing work performed in Pass 1, resulting in PCY being roughly 1.5–2× slower than A-Priori at all scales.

The limited filtering effectiveness is a direct consequence of the characteristics of this dataset: with only 2709 unique actors and baskets of exactly 4 items, the number of candidate pairs is inherently small. Pass 2 in A-Priori is already very fast, so the savings from bitmap filtering are negligible. PCY would show a clear advantage on datasets with a much larger number of unique items, where the number of candidate pairs in Pass 2 becomes the main bottleneck and filtering even a moderate fraction of them would translate into significant time and memory savings.

Note that the reported runtimes may vary slightly across runs due to variability in the execution environment, and the filtering rate may also vary slightly due to Python’s hash randomization. The relative ordering between the two algorithms, however, remains consistent across all runs.

7 Conclusion

This project applied market-basket analysis to the IMDB Top 1000 Movies dataset, treating each movie as a basket and its actors as items. Two algorithms were implemented and compared: A-Priori and PCY.

The A-Priori algorithm successfully identified 299 frequent itemsets, including 25 frequent actor pairs and 3 frequent triples — all corresponding to well-known film franchise casts (Harry Potter, Lord of the Rings, Star Wars). The monotonicity pruning proved highly effective, reducing the number of candidate pairs from over 3.6 million to just 622. From the frequent itemsets, 59 association rules were extracted, all showing high interest values that confirm genuine collaborative patterns between actors.

PCY produced identical results to A-Priori, confirming its correctness, but was slower on this dataset due to the small number of unique actors and the limited filtering effectiveness of the bitmap (1.4% with 1000 buckets). PCY would be the preferred choice on datasets with a much larger item space, where the number of candidate pairs in Pass 2 becomes the main bottleneck and filtering them would translate into significant time and memory savings.

Both algorithms scale linearly with dataset size, demonstrating that the solution is suitable for larger datasets.